

```

break;

case 2: strcpy(root,argv[1]);
        if(strlen(root)-1!='/')
            strcat(root,"/");
        puts(root);

        break;

        default: printf("Ignoring all other arguments after the
first\n");
    }

/* Set up logger*/
fp_log = fopen("inotify_logger.log","a");
if (fp_log == NULL)
{
    printf("Error opening logger. All output will be redirected
to the stdout\n");
    fp_log = stdout;
}

fd = inotify_init();
if ( fd < 0 ) {
    perror( "Couldn't initialize inotify");
}

/* Read the sub-directories at one level under argv[1]
 * and monitor them for access */
add_watches(fd,root);

/* do it forever*/
while(1)
{
    i = 0;
    length = read( fd, buffer, BUF_LEN );

    if ( length < 0 ) {
        perror( "read" );
    }

    /* Read the events*/
    while ( i < length ) {
        struct inotify_event *event = ( struct inotify_event * )
&buffer[ i ];
        if ( event->len ) {
            if ( event->mask & IN_CREATE ) {
                if (event->mask & IN_ISDIR)
                    fprintf(fp_log, "%d DIR::%s CREATED\n", event-
>wd,event->name );
                else
                    fprintf(fp_log, "%d FILE::%s CREATED\n", event->wd,
event->name);
            }
        }
    }
}

```

```

        if ( event->mask & IN_MODIFY ) {
            if (event->mask & IN_ISDIR)
                fprintf(fp_log, "%d DIR::%s MODIFIED\n", event-
>wd,event->name );
            else
                fprintf(fp_log, "%d FILE::%s MODIFIED\n", event-
>wd,event->name );
        }

        if ( event->mask & IN_DELETE ) {
            if (event->mask & IN_ISDIR)
                fprintf(fp_log, "%d DIR::%s DELETED\n", event-
>wd,event->name );
            else
                fprintf(fp_log, "%d FILE::%s DELETED\n", event-
>wd,event->name );
        }

        i += EVENT_SIZE + event->len;
    }
}

/* Clean up*/
( void ) close( fd );

return 0;
}

```

Auto-compiling files

Now that you have a basic idea of how you can use Inotify in applications that may need to monitor the filesystem, let us do something fun, and possibly, quite useful. Using Inotify, let us monitor a directory, and whenever a C source file is created or modified in it, let's invoke 'gcc' to compile the file automatically, and place the binaries in a separate *bin/* directory under this directory. The source code is in Listing 3:*auto_compile.c*:

Listing 3:

```

/* auto_compile.c*/

/* Automatic compilation of C code using Inotify
 *
 * Assumptions:
 * argv[1]: where the source codes are stored
 * bin/ sub-directory: in which the binaries will be stored
 *
 */

#include <stdio.h>
#include <stdlib.h>

```